

AI Assistant — React Migration Setup Guide

Date: 2026-08-25 **Author:** Piranav (AIOS Architect) **Applies to:** LEDSone Staff Dashboards — migrating from plain HTML/CSS/JS + Vercel Serverless to React + proper server + PostgreSQL

Overview

The AI assistant brain (Groq model chain, SQL queries, two-stage flow, chat history) does **not change** during migration. Only the wrapper changes — HTML widget becomes a React component, Vercel functions become proper server routes.

Step 1 — Project Structure (set this up first)

```
src/
├─ components/
│   └─ AiAssistant/
│       ├── AiAssistant.jsx    ← main widget component
│       ├── AiAssistant.css    ← blue UI styles
│       └─ index.js           ← export
├─ layouts/
│   └─ StaffLayout.jsx        ← wraps all staff pages
├─ lib/
│   └─ groq.js                ← copy exact same file from current project
server/
├─ routes/
│   └─ ai/
│       ├── chat.js           ← handles POST /api/ai/chat
│       ├── history.js        ← handles GET /api/ai/history
│       └─ save.js            ← handles POST /api/ai/save
├─ db/
│   ├── pool.js               ← single shared pg.Pool for whole server
│   └─ chatHistory.js         ← chat table queries
├─ config/
│   └─ staffConfig.js         ← all 11 staff campaign IDs + prompts
```

Step 2 — Copy These Exactly (no changes needed)

From the current project, copy these files **as-is** into the new project:

Current File	Copy To	Why
lib/groq.js	src/lib/groq.js	Model chain is identical
SJ_CHAT_DB_URL table schema	server/db/chatHistory.js	Same table structure

The AI logic does not change. Only the wrapper changes.

Step 3 — Build the Shared pg.Pool (do this once)

In the current project, a new Pool is created **per request** — wasteful. In a proper server, create it once at startup and reuse:

```
// server/db/pool.js
const { Pool } = require('pg');

const businessPool = new Pool({
  connectionString: process.env.DATABASE_URL,
  ssl: { rejectUnauthorized: false },
  max: 10,
});

const chatPool = new Pool({
  connectionString: process.env.SJ_CHAT_DB_URL,
  ssl: { rejectUnauthorized: false },
  max: 5,
});

module.exports = { businessPool, chatPool };
```

Every AI route imports from here — no more `new Pool()` inside individual handlers.

Step 4 — Build the API Routes

One route file per action. Example for chat:

```
// server/routes/ai/chat.js
const { businessPool } = require('../../db/pool');
const { callGroqAI } = require('../../lib/groq');
const STAFF_CONFIG = require('../../config/staffConfig');

module.exports = async (req, res) => {
  const { member } = req.params; // e.g. 'theekshy'
  const { message, history } = req.body;

  const config = STAFF_CONFIG[member]; // campaign IDs, market, queries, prompt builder
  if (!config) return res.status(404).json({ ok: false, error: 'Unknown member' });

  const [{ rows: campRows }, { rows: wasteRows }] = await Promise.all([
    businessPool.query(config.campaignQuery, [config.campaignIds]),
    businessPool.query(config.wasteQuery, [config.campaignIds]),
  ]);

  const systemPrompt = config.buildPrompt(campRows, wasteRows);
  const messages = [
    { role: 'system', content: systemPrompt },
    ...history,
    { role: 'user', content: message || 'Assign my tasks for today.' }
  ];
```

```

const result = await callGroqAI(messages);
if (!result.ok) return res.status(502).json({ ok: false, error: result.error });
return res.json({ ok: true, message: result.text });
};

```

Step 5 — Staff Config File (replaces hard-coded IDs)

Instead of campaign IDs scattered across two files (`members-api.js` and `requirement.js`), centralise everything in one place:

```

// server/config/staffConfig.js
module.exports = {
  theekshy: {
    market: 'DE',
    campaignIds: [23714290257, 23684837882],
    campaignQuery: `SELECT ... WHERE campaign_id = ANY($1)`,
    wasteQuery: `SELECT ... WHERE campaign_id = ANY($1)`,
    buildPrompt: (campRows, wasteRows) => `You are Theekshy's AI at LEDSone DE...`,
  },
  thivajini: {
    market: 'FR',
    campaignIds: [23103582865, 23533025729, 23405519670],
    campaignQuery: `SELECT ... WHERE campaign_id = ANY($1)`,
    wasteQuery: `SELECT ... WHERE campaign_id = ANY($1)`,
    buildPrompt: (campRows, wasteRows) => `You are Thivajini's AI at LEDSone FR...`,
  },
  thasitha: {
    market: 'DE',
    campaignIds: null, // dynamic - fetched at runtime
    campaignQuery: `SELECT campaign_id FROM google_ads.campaigns WHERE group_name = 'Thasi'`,
    wasteQuery: `SELECT ... WHERE campaign_id = ANY($1)`,
    buildPrompt: (campRows, wasteRows) => `You are Thasitha's AI at LEDSone DE...`,
  },
  jefri: {
    market: 'DE+IT',
    campaignIds: ['23141810147', '23411228109', '22539594891', '23473840779', '23340277562'],
    buildPrompt: (campRows, wasteRows) => `You are Jefri's AI at LEDSone DE+IT...`,
  },
  mahima: {
    market: 'DE',
    campaignIds: ['20763699505', '23684789991', '23053104908', '23431543574', '23926509987'],
    buildPrompt: (campRows, wasteRows) => `You are Mahima's AI at LEDSone DE...`,
  },
  // kamsi, sukirtha, hethesha, sonya, sajeepan, dilaksi...
};

```

Adding a new staff member = add one entry here only. No touching route files, no touching components.

Step 6 — Build the React Component

```
// src/components/AiAssistant/AiAssistant.jsx
import { useState, useRef, useEffect } from 'react';
import './AiAssistant.css';

export default function AiAssistant({ member }) {
  const [isOpen, setIsOpen] = useState(false);
  const [messages, setMessages] = useState([]);
  const [input, setInput] = useState('');
  const [isLoading, setIsLoading] = useState(false);
  const [briefLoaded, setBriefLoaded] = useState(false);
  const historyRef = useRef([]);

  const loadBrief = async () => {
    // 1. Check today's history first (session restore)
    const hist = await fetch(`/api/ai/history/${member}`).then(r => r.json());
    if (hist.ok && hist.messages.length > 0) {
      setMessages(hist.messages);
      historyRef.current = hist.messages;
      setBriefLoaded(true);
      return;
    }
    // 2. No history today – fetch fresh task card
    setIsLoading(true);
    try {
      const res = await fetch(`/api/ai/chat/${member}`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ message: '', history: [] })
      }).then(r => r.json());

      if (res.ok) {
        setMessages([{ role: 'assistant', content: res.message }]);
        historyRef.current = [{ role: 'assistant', content: res.message }];
        setBriefLoaded(true);
      }
    } catch (e) {
      setMessages([{ role: 'assistant', content: '🔴 Looks like we lost the signal! Check your connection and hit ↻ Refresh.' }]);
    } finally {
      setIsLoading(false);
    }
  };

  const handleOpen = () => {
    setIsOpen(o => !o);
    if (!briefLoaded) loadBrief();
  };
};
```

```

const handleRefresh = () => {
  setBriefLoaded(false);
  setMessages([]);
  historyRef.current = [];
  loadBrief();
};

const sendMessage = async () => {
  if (!input.trim() || isLoading) return;
  const msg = input.trim();
  setInput('');
  setMessages(m => [...m, { role: 'user', content: msg }]);
  setIsLoading(true);
  try {
    const res = await fetch(`/api/ai/chat/${member}`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ message: msg, history: historyRef.current.slice(-6) })
    }).then(r => r.json());

    if (res.ok) {
      setMessages(m => [...m, { role: 'assistant', content: res.message }]);
      historyRef.current = [
        ...historyRef.current,
        { role: 'user', content: msg },
        { role: 'assistant', content: res.message }
      ];
      // save to DB
      await fetch(`/api/ai/save/${member}`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify([
          { role: 'user', content: msg },
          { role: 'assistant', content: res.message }
        ])
      });
    } else {
      setMessages(m => [...m, { role: 'assistant', content: '💡 I\'ve used up all my thinking power for today! The AI quota resets at midnight. 🌙' }]);
    }
  } catch (e) {
    setMessages(m => [...m, { role: 'assistant', content: '📶 Looks like we lost the signal! Check your connection and try again. 📶' }]);
  } finally {
    setIsLoading(false);
  }
};

return (
  <>
    { /* Floating button */ }
    <button className="ai-btn" onClick={handleOpen}>

```

```


    <span className="ai-badge">AI</span>
  </button>

  {/* Panel */}
  {isOpen && (
    <div className="ai-panel">
      <div className="ai-panel-head">
        <span> AI Assistant</span>
        <button onClick={handleRefresh}>↻ Refresh</button>
        <button onClick={() => setIsOpen(false)}>✕</button>
      </div>
      <div className="ai-messages">
        {messages.map((m, i) => (
          <div key={i} className={`ai-msg ${m.role === 'user' ? 'user' : 'bot'}`}>
            {m.content}
          </div>
        ))}
        {isLoading && (
          <div className="ai-thinking">
            <span/><span/><span/>
          </div>
        )}
      </div>
      <div className="ai-input-bar">
        <textarea
          value={input}
          onChange={e => setInput(e.target.value)}
          onKeyDown={e => { if (e.key === 'Enter' && !e.shiftKey) { e.preventDefault();
sendMessage(); } }}
          placeholder="Ask me anything..."
        />
        <button onClick={sendMessage} disabled={!input.trim() || isLoading}>
          ➤
        </button>
      </div>
    </div>
  )}
</>
);
}

```

Step 7 — Add to Staff Pages

Option A — Add to existing page (one page at a time)

If the staff page is already built in React without the AI widget, just add one line:

```

// pages/theekshy.jsx (already built – no refactoring needed)
import AiAssistant from '@components/AiAssistant';

```

```
export default function TheekshyDashboard() {
  return (
    <>
      <YourExistingDashboardContent />
      <AiAssistant member="theekshy" /> { /* add this */}
    </>
  );
}
```

The widget is `position: fixed` (bottom-right) so it floats over the page — zero layout impact.

Option B — Shared layout (covers ALL pages at once)

If most pages are already built or you want automatic coverage:

```
// layouts/StaffLayout.jsx
import AiAssistant from '@components/AiAssistant';

export default function StaffLayout({ member, children }) {
  return (
    <>
      {children}
      <AiAssistant member={member} />
    </>
  );
}

// pages/theekshy.jsx
export default function TheekshyDashboard() {
  return (
    <StaffLayout member="theekshy">
      <YourExistingDashboardContent />
    </StaffLayout>
  );
}
```

Situation	Use
A few pages already built	Option A — add one line per page
Most pages already built	Option B — shared layout, one change covers all
Building from scratch	Option B — design layout with AI built in from day one

Step 8 — Environment Variables (same as now — nothing new)

```
GROQ_API_KEY=gsk_...           # all 12 staff (Groq chain + preference analysis)
NVIDIA_API_KEY=nvapi-...       # Muguntha only (NVIDIA NIM, falls back to Groq if
missing)
DATABASE_URL=postgres://...    # business data — Google Ads, Shopify, merchant products
```

```

SJ_CHAT_DB_URL=postgresql://...    # chat history – all 12 *_ai_chat tables (incl.
muguntha_ai_chat)
FEED_TRACKER_DB_URL=postgresql://... # requirement trackers – Muguntha AI only
EOD_GITHUB_TOKEN=ghp...            # GitHub PAT – Muguntha EOD checks only

```

What Changes vs What Stays the Same

Part	Current	React + Server	Changes?
Groq model chain	lib/groq.js	lib/groq.js (copied)	No
NVIDIA model	lib/nvidia.js (Muguntha only)	lib/nvidia.js (copied)	No
SQL queries	Inside handler functions	Same SQL, same Promise.all	No
Campaign IDs	Scattered in 2 files	staffConfig.js (centralised)	Structure only
Chat DB tables	SJ_CHAT_DB_URL	Same DB, same tables	No
Chat table role values	user, assistant	user, assistant, preference	Add preference
Daily reset logic	session_date = CURRENT_DATE	Same filter	No
Session restore	Parallel (race condition risk)	Async/await sequential	Fix on port
Preference learning	analyseAndSavePreference() per handler	Port to server/lib/preference.js	Centralise
System prompt structure	INSTRUCTIONS only	URGENCY RANKING + BEHAVIOUR RULES + emoji flags	Port as-is
Time-aware greeting	new Date().getHours() per handler	Same — compute server-side	No
pg.Pool	Created per request	Created once at server start	Better
Widget JS	Copy-pasted in 12 HTML files	One React component	Much better
Adding new staff	Edit 2 API files + 1 HTML file	Add 1 entry to staffConfig.js	Much better

Order of Work When You Start

Day 1

- └─ Step 1: Set up folder structure (30 min)
- └─ Step 2: Copy lib/groq.js (5 min)
- └─ Step 3: Build server/db/pool.js (30 min)
- └─ Step 4: Build server/config/staffConfig.js (2 hrs – port all 11 staff)

Day 2

- └─ Step 5: Build 3 API routes (chat / history / save) (1-2 hrs)
- └─ Step 6: Build AiAssistant React component (2-3 hrs)
- └─ Step 7: Add to StaffLayout or per page (30 min)

Total: one focused day per section, or two days end-to-end.

Key Rules to Carry Over

1. **Always use `pg.Pool`, never `pg.Client`** — AI handlers run parallel queries with `Promise.all`. `pg.Client` crashes on concurrent queries.
2. **Daily reset is automatic** — filter chat history by `session_date = CURRENT_DATE`. No cron job needed.
3. **Session restore is `async/await sequential`** — await the history check first. If history exists, return immediately — do NOT fire the AI call. See Section 20 of `AI-ASSISTANT-WORKFLOW.md` for the correct pattern.
4. **Thasitha uses dynamic campaign query** — `WHERE group_name = 'Thasi'`, not a hard-coded ID array. Keep this pattern — it auto-picks new campaigns.
5. **Kamsi has extra body fields** — `metaSummary` and `prioritySummary` must be passed in the fetch body from the Kamsi page component.
6. **Chat table has 3 role values** — `user`, `assistant`, and `preference`. The preference row is written by the preference learning system on new-day detection. When reading history to restore a session, filter to `role IN ('user', 'assistant')` only — exclude preference rows from the chat display.
7. **Preference learning must be ported** — the `analyseAndSavePreference` + `getLatestPreference` helpers must be included in the React server. On new-day detection (today's DB empty), run the analysis before building the system prompt. Inject result as `LEARNED FROM PREVIOUS SESSIONS` block.
8. **System prompts include urgency ranking + behaviour rules** — every staff prompt now has `URGENCY RANKING`, `BEHAVIOUR RULES`, emoji priority flags (🔴 🟡 🟢), and a follow-up rule (end with one clear action). Port these from the current `members-api.js` / `requirement.js` — do not use the old plain `INSTRUCTIONS` format.
9. **Time-aware greeting** — never hardcode "Good morning". Always compute: `const hr = new Date().getHours(); const greeting = hr < 12 ? 'Good morning' : hr < 17 ? 'Good afternoon' : 'Good evening';` server-side before building the system prompt.
10. **Muguntha uses NVIDIA, not Groq** — `lib/nvidia.js` with `callNvidiaAI()`. Falls back to Groq if `NVIDIA_API_KEY` not set. Her preference analysis also uses NVIDIA (not Groq).

Full Reference

Full workflow documentation: `workflows/AI-ASSISTANT-WORKFLOW.md`

Last updated: 2026-08-25 (updated — preference learning, urgency ranking, behaviour rules, time-aware greeting) Built by: Piranav (AIOS Architect) + Claude Sonnet 4.6